```
}).then((response) => {
    return response.text();
}).then((result) => {
    alert(`Your stay will cost ${result}`);
});
```

If you return a complex type, the value will be serialized to JSON, and the content type will be set to `application/json`:

```
var totalCost = bookingService.Calculate(booking);
return new { TotalCost = totalCost };
```

These options might be handy in some cases, but they only save a few keystrokes compared to using one of the `IResult` options, which are strongly typed and lend themselves to being tested.

ROUTE HANDLER AUTHORIZATION

Route handlers can take part in authorization along with your pages. You can pass the `Authorize` attribute in before any parameters. For example,

```
app.MapPost("/api/property/booking", [Authorize](BookingInputModel model) =>
```

Alternatively, you can chain the `RequireAuthorization` method to the handler:

```
app.MapPost("/api/property/booking", (BookingInputModel model) => {
    ...
}).RequireAuthorization();
```

The `RequireAuthorization` method takes a `params string[]` as a parameter, enabling you to pass in the name or names of any authorization policies that should be applied to the route handler.